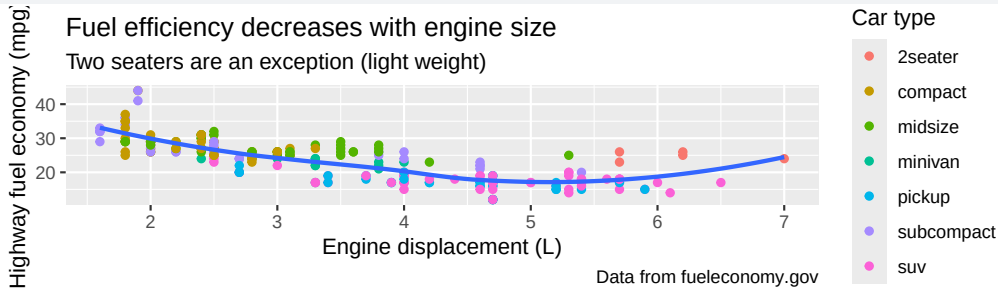


R4DS 07 – Communication

Labels

labs() — Add Context to Plots

```
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point(aes(color = class)) +  
  geom_smooth(se = FALSE) +  
  labs(  
    title = "Fuel efficiency decreases with engine size",  
    subtitle = "Two seaters are an exception (light weight)",  
    caption = "Data from fueleconomy.gov",  
    x = "Engine displacement (L)",  
    y = "Highway fuel economy (mpg)",  
    color = "Car type"  
  )
```



Best Practices for Labels

- **Title:** summarize the main **finding**, not the plot type
- **Subtitle:** additional context in smaller font
- **Caption:** data source, credits
- **Axis labels:** descriptive names with units
- **Legend title:** clear variable name via `labs(color = ...)`

Math labels: use `quote()` instead of `""`:

```
labs(x = quote(x[i]), y = quote(sum(x[i]^2, i == 1, n)))
```

Exercises (Labels)

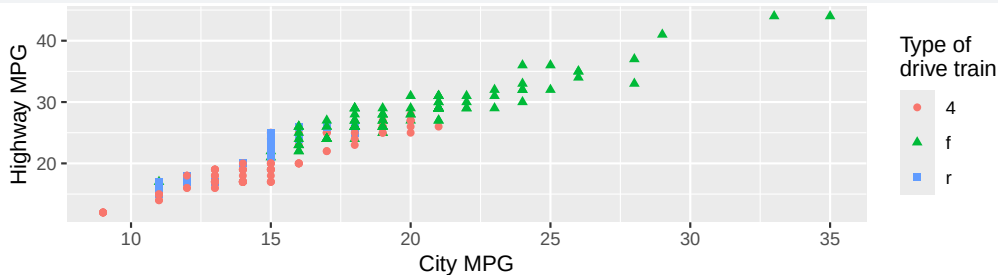
- 1 Create a plot of fuel economy data with customized `title`, `subtitle`, `caption`, `x`, `y`, and `color` labels.
- 2 Recreate: scatterplot of `cty` vs. `hwy`, colored and shaped by `drv`, with combined legend label "Type of drive train".
- 3 Take an exploratory plot and add informative titles.

Solutions (Labels)

① (Open-ended — use `labs()` with all six arguments.)

②

```
ggplot(mpg, aes(x = cty, y = hwy, color = drv, shape = drv)) +  
  geom_point() +  
  labs(x = "City MPG", y = "Highway MPG",  
       color = "Type of\ndrive train",  
       shape = "Type of\ndrive train")
```

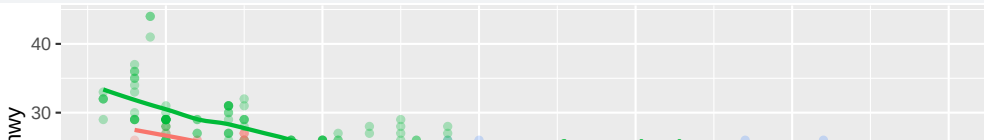


Annotations

Direct Labels with geom_text()

```
label_info <- mpg |>
  group_by(drv) |>
  arrange(desc(displ)) |>
  slice_head(n = 1) |>
  mutate(drive_type = case_when(
    drv == "f" ~ "front-wheel",
    drv == "r" ~ "rear-wheel",
    drv == "4" ~ "4-wheel"
  ))

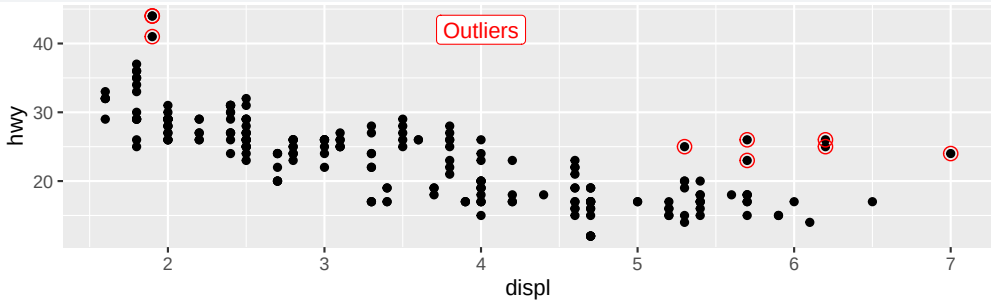
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +
  geom_point(alpha = 0.3) + geom_smooth(se = FALSE) +
  geom_text(data = label_info,
    aes(label = drive_type),
    fontface = "bold", size = 4, hjust = "right") +
  theme(legend.position = "none")
```



Labeling Outliers

```
potential_outliers <- mpg |>
  filter(hwy > 40 | (hwy > 20 & displ > 5))

ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  geom_point(data = potential_outliers,
            color = "red", size = 3, shape = "circle open") +
  annotate(geom = "label", x = 4, y = 42,
         label = "Outliers", color = "red")
```



Function	Purpose
<code>geom_text()</code> / <code>geom_label()</code>	Label points from data
<code>annotate()</code>	Add single annotations (text, arrows, rectangles)
<code>geom_hline()</code> / <code>geom_vline()</code>	Reference lines
<code>geom_segment(arrow = ...)</code>	Arrows pointing to features
<code>geom_rect()</code>	Highlight rectangular regions

`annotate()` is for one-off elements; geoms are for data-driven labels.

Exercises (Annotations)

- 1 Use `geom_text()` with infinite positions to place text at the four corners.
- 2 Use `annotate()` to add a point geom in the middle of a plot.
- 3 How do `geom_text()` labels interact with faceting? How to label a single facet?
- 4 What arguments to `geom_label()` control the background box?
- 5 What are the four arguments to `arrow()`?

❶

```
ggplot(mpg, aes(x = displ, y = hwy)) + geom_point() +  
  geom_text(aes(x = x, y = y, label = label), data = tibble(  
    x = c(-Inf, Inf, -Inf, Inf),  
    y = c(-Inf, -Inf, Inf, Inf),  
    label = c("BL", "BR", "TL", "TR")),  
    hjust = c(0, 1, 0, 1), vjust = c(0, 0, 1, 1))
```

❷ `annotate(geom = "point", x = 3.5, y = 25, size = 5, color = "red")`

❸ Labels appear in all facets by default. To label one facet, pass a data frame with the faceting variable set to the desired level.

- 4 `label.padding`, `label.r` (corner radius), `label.size` (border width), `fill`, `color`.
- 5 `angle`, `length`, `ends` (“last”, “first”, “both”), `type` (“open”, “closed”).

Scales

Scale Naming Convention

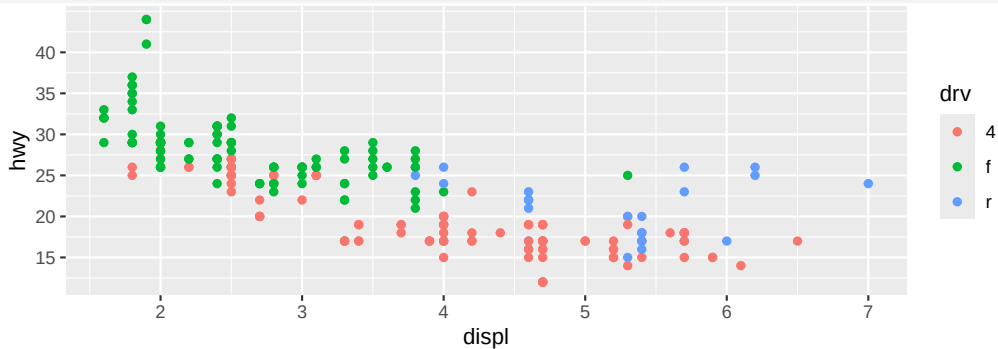
```
# ggplot2 adds these defaults automatically:  
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point(aes(color = class)) +  
  scale_x_continuous() +  
  scale_y_continuous() +  
  scale_color_discrete()
```

Pattern: **scale_** + aesthetic + **_** + type.

Override defaults to customize **breaks**, **labels**, or the entire scale.

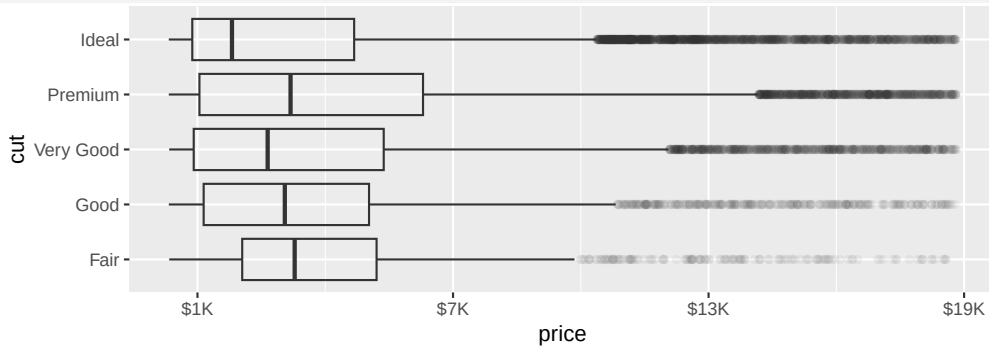
Custom Breaks and Labels

```
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +  
  geom_point() +  
  scale_y_continuous(breaks = seq(15, 40, by = 5))
```



Formatting with scales Package

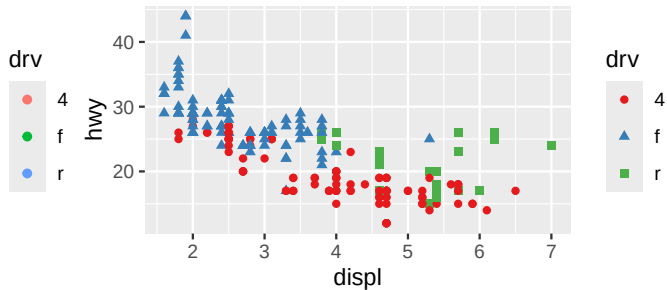
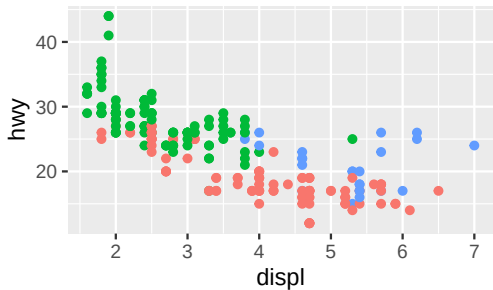
```
ggplot(diamonds, aes(x = price, y = cut)) +  
  geom_boxplot(alpha = 0.05) +  
  scale_x_continuous(  
    labels = label_dollar(scale = 1/1000, suffix = "K"),  
    breaks = seq(1000, 19000, by = 6000)  
  )
```



Also: `label_percent()`, `label_comma()`, `label_number()`.

Color Scales

```
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point(aes(color = drv))  
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point(aes(color = drv, shape = drv)) +  
  scale_color_brewer(palette = "Set1")
```



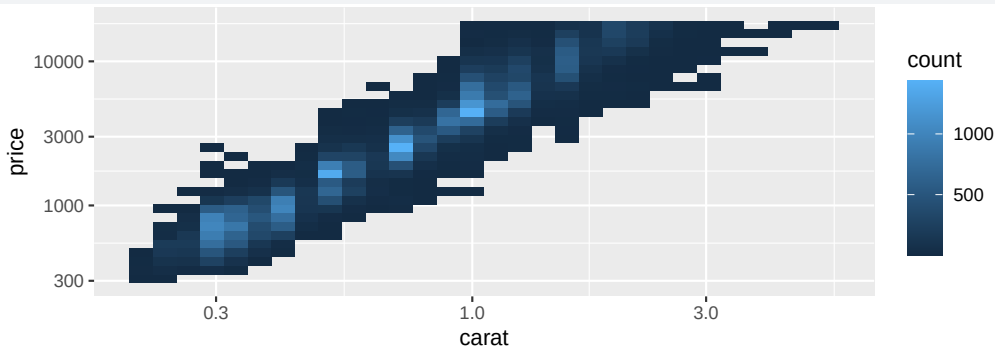
Left: default colors. Right: ColorBrewer palette + redundant shape (colorblind-friendly).

```
# Manual: predefined value-to-color mapping
scale_color_manual(values = c(
  Republican = "#E81B23", Democratic = "#00AEF3"))

# Continuous: viridis (perceptually uniform, colorblind-safe)
scale_fill_viridis_c()    # continuous
scale_fill_viridis_d()    # discrete
scale_fill_viridis_b()    # binned
```

Log Scales

```
ggplot(diamonds, aes(x = carat, y = price)) +  
  geom_bin2d() +  
  scale_x_log10() + scale_y_log10()
```



Transform in the scale (not `aes()`) → axes labeled on original scale.

Zooming: Three Approaches

```
# 1. Filter data (changes smooth curve!)
mpg |> filter(displ >= 5 & displ <= 6) |>
  ggplot(aes(x = displ, y = hwy)) + geom_smooth()

# 2. Set scale limits (drops data outside range)
scale_x_continuous(limits = c(5, 6))

# 3. coord_cartesian (just zooms, keeps all data)
coord_cartesian(xlim = c(5, 6), ylim = c(10, 25))
```

Best practice: use `coord_cartesian()` to zoom. Use `limits` to match scales across plots.

Exercises (Scales)

- ❶ Why doesn't `scale_color_gradient()` work on a `geom_hex()` plot?
- ❷ What is the first argument to every scale? How does it compare to `labs()`?
- ❸ Customize the presidential terms plot: colors, breaks, labels, y-axis.
- ❹ Use `override.aes` to make a legend with `alpha = 1/20` points easier to see.

Solutions (Scales)

- 1 `geom_hex()` uses the `fill` aesthetic, not `color`. Need `scale_fill_gradient()`.
- 2 The first argument is `name` (the axis/legend title) — same as `labs()`.
- 3

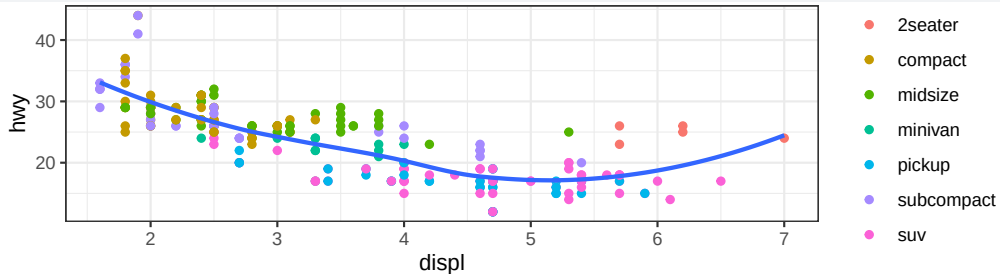
```
presidential |>
  mutate(id = 33 + row_number()) |>
  ggplot(aes(x = start, y = id, color = party)) +
  geom_point() + geom_segment(aes(xend = end, yend = id)) +
  scale_color_manual(values = c(
    Republican = "red", Democratic = "blue")) +
  scale_x_date(breaks = presidential$start,
               date_labels = "'%y")
```

- 4 Add `guides(color = guide_legend(override.aes = list(alpha = 1, size = 3)))`.

Themes

Built-in Themes

```
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point(aes(color = class)) +  
  geom_smooth(se = FALSE) +  
  theme_bw()
```



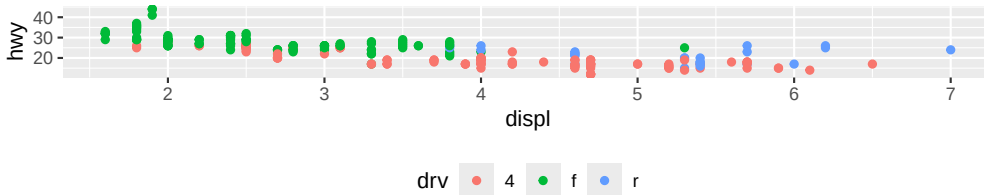
Theme	Description
<code>theme_gray()</code>	Default (gray background)
<code>theme_bw()</code>	White background, grid lines
<code>theme_minimal()</code>	Minimal, no background
<code>theme_classic()</code>	Axes only, no grid
<code>theme_light()</code>	Light axes and grid
<code>theme_dark()</code>	Dark background
<code>theme_void()</code>	Nothing — just geoms

More themes: **ggthemes** package.

Customizing Theme Elements

```
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +  
  geom_point() +  
  labs(title = "Engine size vs. fuel economy",  
        caption = "Source: fueleconomy.gov") +  
  theme(  
    plot.title = element_text(face = "bold"),  
    plot.title.position = "plot",  
    plot.caption = element_text(hjust = 0),  
    legend.position = "bottom"  
  )
```

Engine size vs. fuel economy



Source: fueleconomy.gov

- 1 Pick a theme from **ggthemes** and apply it.
- 2 Make axis labels blue and bolded.

Solutions (Themes)

1

```
library(ggthemes)
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() + theme_economist()
```

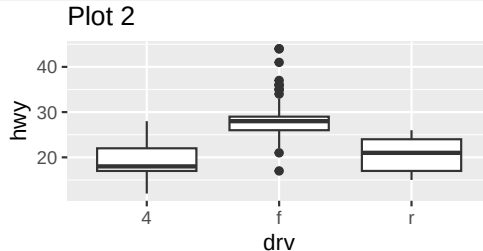
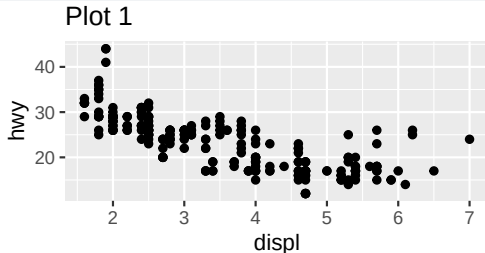
2

```
theme(axis.title = element_text(color = "blue", face = "bold"))
```

Layout

Combining Plots with patchwork

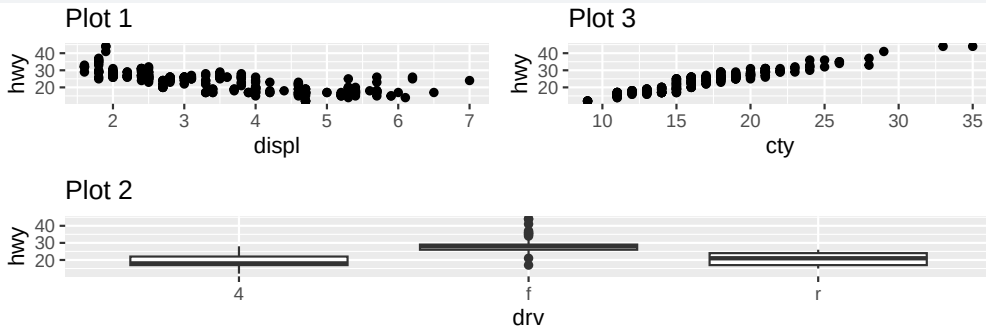
```
p1 <- ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point() + labs(title = "Plot 1")  
p2 <- ggplot(mpg, aes(x = drv, y = hwy)) +  
  geom_boxplot() + labs(title = "Plot 2")  
p1 + p2
```



+ places plots side by side. Load **patchwork** to enable this.

Complex Layouts

```
p3 <- ggplot(mpg, aes(x = cty, y = hwy)) +  
  geom_point() + labs(title = "Plot 3")  
(p1 | p3) / p2
```



| = side by side, / = stack vertically, parentheses group.

```
# Shared legend and common title
(p1 + p2) +
  plot_annotation(
    title = "Combined plots",
    caption = "Source: mpg dataset"
  ) +
  plot_layout(guides = "collect") &
  theme(legend.position = "bottom")
```

- `plot_annotation()`: shared title, subtitle, caption
- `plot_layout(guides = "collect")`: one combined legend
- `&`: apply theme to **all** subplots

- 1 What happens if you omit parentheses in $(p1 \mid p3) / p2$?

- 1 Without parentheses, $p1 \mid p3 / p2$ evaluates $p3 / p2$ first (stacks them), then places $p1$ next to the stack. Parentheses control grouping, just like in math.